

Field	Your Data
Workstation / Laptop	MacBook Air 13-inch (MacBookAir10,1)
Processor / SoC	Apple M1
CPU topology	4 Performance cores + 4 Efficiency cores
Logical CPU cores	8
L1 Data Cache	64 KB reported by <code>sysctl</code>
L1 Instruction Cache	128 KB reported by <code>sysctl</code>
L2 Cache	4 MB reported by <code>sysctl</code>
Memory	8 GB Unified Memory
Memory type	LPDDR4 reported by macOS; M1 specifications are commonly listed as LPDDR4X-4266
OS	macOS 14.7.6

Q1.1

The major inflection point occurs at **p = 8 workers**. Execution time decreases from 0.5726 s at 4 workers to 0.4632 s at 8 workers. However, increasing the number of workers beyond 8 does not improve performance: execution time increases to 0.4954 s at 12 workers and 0.5275 s at 16 workers.

This corresponds closely to the Apple M1 hardware topology, which has **4 performance cores and 4 efficiency cores, for a total of 8 physical CPU cores and 8 logical CPUs**. Therefore, the benchmark reaches practical physical-core saturation around 8 workers. Beyond this point, additional processes compete for existing CPU resources and introduce scheduling and process-management overhead.

Q1.2

Using the measured values:

$$T(1) = 2.1157 \text{ s}$$

$$T(4) = 0.5726 \text{ s}$$

Therefore:

$$S(4) = T(1) / T(4)$$

$$S(4) = 2.1157 / 0.5726 = 3.6949$$

Using Amdahl's Law:

$$3.6949 = 1 / [(1 - P) + P/4]$$

Taking the reciprocal:

$$0.27064 = 1 - P + P/4$$

$$0.27064 = 1 - 0.75P$$

$$0.75P = 0.72936$$

$$P = 0.9725$$

Therefore, the estimated parallel fraction is **$P \approx 97.25\%$** , while approximately **2.75%** of the workload is non-parallel.

Q1.3

Using $P = 0.9725$:

$$S_{\text{max}} = 1 / (1 - P)$$

$$S_{\text{max}} = 1 / (1 - 0.9725)$$

$$S_{\text{max}} \approx \mathbf{36.33\times}$$

This means that even with an unlimited number of processors, this workload could theoretically achieve only about 36.33× speedup because of the approximately 2.75% serial portion. Therefore, a 128-core server would not provide a 128× speedup for this workload. In practice, the achievable speedup would be even lower because of scheduling, communication, process-management, and resource-contention overhead.

Q2.1

A 64-bit pointer or reference occupies:

$$64 \text{ bits} / 8 = \mathbf{8 \text{ bytes}}$$

Therefore, 16 consecutive references occupy:

$$16 \times 8 = \mathbf{128 \text{ bytes}}$$

Since one cache line is 64 bytes:

$$128 \text{ bytes} > 64 \text{ bytes}$$

Thus, an index stride of 16 separates the accessed references by 128 bytes, which is larger than one 64-byte cache line. Therefore, the padded indices cannot reside within the same 64-byte cache line, assuming the relevant reference layout is aligned normally.

Q2.2

Initially, a cache line containing the adjacent elements may be in the Shared state in multiple cores. When Core 0 writes to index 0, it must obtain exclusive ownership of the cache line. The line therefore becomes Modified in Core 0, while copies in other cores become Invalid.

When Core 1 subsequently writes to index 1, it needs the same cache line because index 0 and index 1 are located close together. A coherence request causes Core 0's copy to be invalidated, and Core 1 obtains the cache line in a writable state.

If Core 0 writes again, ownership must move back to Core 0. Repeated ownership transfers create cache-coherence traffic and additional latency. This is the fundamental mechanism of false sharing: logically independent variables cause unnecessary cache-line transfers because they occupy the same cache line.

Q2.3

One industry-standard mechanism is **C++11 `alignas(64)`**, which allows data structures or variables to be aligned to a cache-line boundary. Additional padding can then ensure that independently modified data occupies separate cache lines.

Another mechanism is **Java `@Contended`**, where the JVM adds padding around highly contended fields to prevent multiple frequently modified fields from sharing the same cache line.

Go can also use explicit structure padding to separate frequently modified fields and reduce false sharing.

Q3.1

In my empirical run, the UnsafeCounter produced:

2,000,000 / 2,000,000

Therefore, the observed corrupted count was **0**, and the defect count was **0** in this particular execution. The result was correct even though the implementation was logically unsafe.

The increment operation can conceptually be represented as:

LOAD → ADD 1 → STORE

If two threads perform these operations concurrently, both may load the same old value, add one independently, and then store the same new value. In that situation, one increment can be lost.

However, in this particular Python 3.13 execution, the GIL and execution behavior prevented the benchmark from producing an incorrect result. Therefore, the empirical result was correct, but the implementation should still be considered unsafe as a general concurrent design because it does not explicitly guarantee synchronization.

Q3.2

The lockless implementation uses a Map-Reduce approach. Each thread maintains its own private counter, so threads do not continuously modify shared mutable state. After all threads finish, the main thread performs a single reduction by summing the local results.

The measured result was:

Lockless Value = 2,000,000
Execution Time = 0.0700 s

The LockedCounter required 0.2632 s.

Speedup:

$$0.2632 / 0.0700 = \mathbf{3.76\times}$$

Therefore, the lockless implementation achieved approximately **3.76× speedup over the LockedCounter**, exceeding the required 2× improvement.

Q3.3

Eliminating shared mutable state is more scalable than optimizing lock primitives because it removes the main source of contention instead of simply making contention cheaper.

With a shared counter, threads repeatedly compete for the same variable and synchronization mechanism. This causes waiting, serialization, and cache-coherence traffic.

With thread-local accumulation, each thread writes only to its own private state. No lock is required during the main computation, and the results are combined only once after all threads finish.

Therefore, the Map-Reduce approach reduces synchronization, cache-coherence traffic, and serialization. This makes it more suitable for high-throughput concurrent systems.

Q4.1

The compute-bound workload shows almost no scaling in elapsed time. One worker required 0.7868 s, while two and four workers required 0.7962 s and 0.8351 s respectively. Thus, the execution time remained approximately constant.

The memory-bound workload behaved differently. One worker required 0.2449 s, two workers required 0.2905 s, and four workers required 0.8106 s. Four workers therefore took approximately 3.31 times longer than one worker.

This demonstrates the memory-wall effect. Memory bandwidth is a finite shared resource. Increasing the number of workers does not create additional DRAM bandwidth. Once the available bandwidth is approached, additional workers mainly increase memory-controller contention and waiting time.

The compute workload is dominated by CPU arithmetic, while the memory workload depends heavily on data movement. Therefore, adding workers to the memory-bound workload causes significantly greater contention and performance degradation.

Q4.2

My MacBook Air uses 8 GB of unified memory. The Apple M1 memory subsystem uses LPDDR4X-4266 memory with a 128-bit memory interface. Its theoretical peak memory bandwidth is approximately **68.25 GB/s**.

A 128-bit interface corresponds to 16 bytes per transfer:

$$16 \text{ bytes} \times 4.266 \text{ billion transfers/second} \approx 68.3 \text{ GB/s}$$

This bandwidth is shared by the components using the unified memory architecture. Therefore, adding more CPU workers cannot increase the physical memory bandwidth beyond the available limit. Once memory bandwidth becomes saturated, additional workers mainly increase contention and waiting time rather than useful throughput.

Q4.3

If the embedding pipeline is already memory-bandwidth saturated, simply adding more CPU OpenMP threads is unlikely to provide substantial speedup because the limiting resource is the memory subsystem rather than the number of CPU execution units.

CUDA GPU kernels could provide greater potential performance if the workload has sufficient parallelism and the GPU provides substantially higher memory bandwidth. However, data-transfer overhead between CPU and GPU memory must also be considered.

Therefore, if profiling confirms that the CPU implementation is already saturating memory bandwidth, increasing CPU parallelism alone is unlikely to solve the bottleneck. A GPU-based

implementation becomes attractive when the workload can effectively exploit GPU parallelism and memory bandwidth.

BONUS TASK

Empirical Curve

The threading implementation remained almost flat across all worker counts. Execution time varied from approximately 2.05 to 2.15 seconds.

The multiprocessing implementation scaled substantially better: execution time decreased from 2.5418 s with one worker to 1.3893 s with two workers and 0.8652 s with four workers. The best measured result was 0.8623 s with eight workers.

Inflection Cliff

The main performance cliff occurs after **8 multiprocessing workers**. At 8 workers, the execution time was 0.8623 s. Increasing the number to 16 workers increased execution time to 1.3373 s, and 32 workers increased it further to 2.1851 s.

This corresponds closely to the Apple M1's **8 physical CPU cores: 4 performance cores and 4 efficiency cores**. Beyond 8 workers, additional processes must compete for already occupied CPU resources, increasing scheduling, context-switching, cache, and resource-contention overhead.

Architectural Autopsy

The threading curve remains virtually flat because the workload consists of CPU-bound Python code executed by CPython. The Global Interpreter Lock (GIL) prevents multiple Python threads from executing Python bytecode simultaneously within the same interpreter.

As a result, increasing the number of threads from 1 to 32 does not provide meaningful CPU parallelism. Additional threads mainly introduce scheduling and interpreter overhead.

Multiprocessing behaves differently because each worker is a separate Python process with its own interpreter and GIL. Therefore, multiple processes can execute CPU-bound Python code simultaneously on different CPU cores.

The results demonstrate this clearly: execution time decreased from 2.5418 s with one process to 0.8623 s with eight processes. After eight workers, performance degraded because the workload exceeded the available physical CPU resources.

Physical-Core Saturation

The measured saturation point is approximately **8 multiprocessing workers**. The Apple M1 contains four performance cores and four efficiency cores, giving a total of eight physical CPU cores.

The benchmark therefore shows a close relationship between the hardware topology and the scaling curve. Up to approximately eight workers, additional CPU resources can be utilized. Beyond eight, additional processes compete for the same physical resources, increasing scheduling and context-switching overhead as well as cache and resource contention.

The performance degradation is visible in the measurements: **0.8623 s at 8 workers, 1.3373 s at 16 workers, and 2.1851 s at 32 workers**.